guarantee that they will respect the requirement of being not closer than T_2 from each other, because centroids from different shards haven't been compared to each other.

It's not yet time to give up, though! Luckily there is an easy solution to this issue, and the solution is still… canopy clustering!

If, in fact, we gather all the centroids from each mapper together, we can apply canopy clustering algorithm again to this new (smaller) dataset, refining the selection and this time guaranteeing that no two points in the output from this second pass will be at a closer distance than T_2: check the last row in figure 13.9 to have an idea of how this second pass works. The solution is also efficient because the size of the new dataset (containing only the centroids produced by the mappers) is orders of magnitude smaller than the original dataset (assuming the distances T_1 and T_2 have been chosen properly), and therefore in step 2 we can spin up a single reducer, and run canopy clustering on a single machine and on all the centroids from step 1.

At this point, we need to make a consideration: when used as a preliminary step for k-means, the algorithm has a slightly different goal (and different output) than when used as a standalone coarse-grained clustering algorithm:

- To k-means, we only have to return a list of centroids;
- The standalone algorithm will also need to return, for each canopy, the points belonging to it.

Therefore, we need to treat these two cases separately.